Object-Oriented Programming in Java

A Complete Beginner's Study Guide

Classes & Objects • Encapsulation • Inheritance • Polymorphism • Abstraction

Prepared as a teaching resource for students new to OOP

Table of Contents

1. Introduction: What Is OOP?

Object-Oriented Programming (OOP) is a way of writing programs by organizing code around "objects" instead of just functions and logic. An object bundles together data (called fields or attributes) and the actions that work on that data (called methods) into a single unit.

Java was built from the ground up as an object-oriented language, so understanding OOP is not optional for a Java developer — it is the foundation everything else is built on: Spring Boot, Android development, enterprise applications, and design patterns all assume you are comfortable thinking in objects.

1.1 Procedural Programming vs. OOP

Before OOP became popular, most programs were written in a procedural style (like plain C). In procedural programming, a program is a list of instructions and functions that operate on data that is often kept separate from the functions themselves.

Aspect	Procedural Programming	Object-Oriented Programming
Main unit	Function / procedure	Object (class instance)
Data & behavior	Kept separate	Bundled together (encapsulation)
Focus	What steps happen	What entities exist and how they interact
Reusability	Limited, via copy/paste or functions	High, via inheritance & composition
Example language	C	Java, C++, C#

1.2 A Real-World Analogy

Think of a "Car". In the real world, every car has attributes (color, brand, speed, fuel level) and behaviors (start, accelerate, brake, stop). In OOP we model this exact idea: the attributes become fields, and the behaviors become methods, all packaged inside a class called Car.

Note: A class is like a blueprint (e.g., the engineering drawing of a car). An object is a real car built from that blueprint. You can build many cars (objects) from one blueprint (class), and each one can have different colors or speeds.

2. Classes and Objects

2.1 What Is a Class?

A class is a template that defines what data (fields) and what actions (methods) its objects will have. Declaring a class does not create anything in memory by itself — it just defines the shape.

```
public class Car {
    // fields (attributes / instance variables)
    String color;
    String brand;
```

```
    int speed;

    // method (behavior)
    void accelerate() {
        speed = speed + 10;
        System.out.println(brand + " is now going " + speed + " km/h");
    }
}
```

2.2 What Is an Object?

An object is a real instance of a class, created in memory using the new keyword. Each object has its own copy of the fields defined in the class.

```
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();      // creating an object (instance)
        myCar.color = "Red";
        myCar.brand = "Toyota";
        myCar.speed = 0;

        myCar.accelerate();          // calling a method on the object
        // Output: Toyota is now going 10 km/h
    }
}
```

Here, Car is the class (the blueprint), and myCar is the object (the actual car). You could create a second object, e.g. Car yourCar = new Car();, and it would have its own independent color, brand, and speed.

2.3 Fields, Methods, and the “this” Keyword

The this keyword refers to the current object — it is used mainly to distinguish between a field and a parameter that share the same name.

```
public class Car {
    String brand;

    void setBrand(String brand) {
        this.brand = brand;   // this.brand = the field, brand = the parameter
    }
}
```

3. Constructors

A constructor is a special method that runs automatically when an object is created with new. It is used to initialize the object's fields. A constructor has the same name as the class and no return type (not even void).

3.1 Default Constructor

If you do not write any constructor, Java automatically provides an empty one for you.

```
public class Car {
    String brand;
    // Java secretly provides: public Car() { }
}
```

3.2 Parameterized Constructor

You can define your own constructor that takes parameters, so objects are created already filled with values.

```
public class Car {
    String brand;
    String color;
    int speed;

    // parameterized constructor
    public Car(String brand, String color) {
        this.brand = brand;
        this.color = color;
        this.speed = 0;
    }
}

// Usage:
Car myCar = new Car("Toyota", "Red");
```

3.3 Constructor Overloading

A class can have multiple constructors with different parameter lists. Java chooses the right one based on the arguments you pass.

```
public class Car {
    String brand;
    String color;

    public Car() {
        this("Unknown", "White");   // calls the other constructor
    }

    public Car(String brand, String color) {
        this.brand = brand;
        this.color = color;
    }
}
```

Note: this(...) calls another constructor in the SAME class. super(...) calls a constructor in the PARENT class. Both, when used, must be the very first line inside the constructor.

4. The Four Pillars of OOP

Every OOP language, including Java, is built around four core principles. Mastering these four ideas is the real goal of learning OOP:

- Encapsulation — hiding internal details and protecting data
- Inheritance — reusing code by building new classes from existing ones
- Polymorphism — one action, many forms
- Abstraction — hiding complexity, showing only what's necessary

4.1 Encapsulation

Encapsulation means bundling data (fields) and the methods that operate on it inside one class, and restricting direct access to that data from outside the class. Instead of letting outside code freely change a field, we make fields private and expose controlled access through public getter and setter methods.

```
public class BankAccount {
    private double balance;   // private: cannot be accessed directly from outside

    public double getBalance() {        // getter
        return balance;
    }

    public void deposit(double amount) {  // setter-like method with validation
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Invalid deposit amount");
        }
    }
}
```

Without encapsulation, any code could set balance = -1000000; directly. With encapsulation, the deposit() method controls exactly how the balance can change, protecting the object from invalid states.

Key Idea: Encapsulation = private fields + public getters/setters (with validation logic when needed).

4.2 Access Modifiers

Access modifiers control which classes can “see” a field, method, or class. They are essential for encapsulation.

Modifier	Same Class	Same Package	Subclass (other package)	Everywhere
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

5. Inheritance

Inheritance allows a new class (the child / subclass) to reuse fields and methods from an existing class (the parent / superclass), using the extends keyword. This models an “is-a” relationship: a Dog is an Animal, a Car is a Vehicle.

```
public class Animal {
    String name;

    public void eat() {
        System.out.println(name + " is eating");
    }
}

public class Dog extends Animal {   // Dog inherits from Animal
    public void bark() {
        System.out.println(name + " is barking");
    }
}

// Usage:
Dog myDog = new Dog();
myDog.name = "Rex";
myDog.eat();    // inherited from Animal -> "Rex is eating"
myDog.bark();   // defined in Dog -> "Rex is barking"
```

5.1 The super Keyword

super refers to the parent class. It is used to call the parent's constructor or to access a parent method that was overridden.

```
public class Animal {
    String name;
    public Animal(String name) {
        this.name = name;
    }
    public void makeSound() {
        System.out.println("Some generic animal sound");
```

```
    }
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name);   // calls Animal's constructor
    }

    @Override
    public void makeSound() {
        super.makeSound();          // calls Animal's version first
        System.out.println("Woof woof!");
    }
}
```

5.2 Method Overriding

Overriding means a subclass provides its own implementation of a method that is already defined in its parent class. The method signature (name + parameters) must be identical. The @Override annotation is optional but strongly recommended — it lets the compiler catch mistakes.

> **Note:** Java does NOT support multiple inheritance of classes (a class cannot extend two classes). This avoids ambiguity problems. Java allows multiple inheritance only through interfaces (see Section 7).

5.3 Types of Inheritance Supported in Java

- Single inheritance — Dog extends Animal
- Multilevel inheritance — Puppy extends Dog extends Animal
- Hierarchical inheritance — Dog and Cat both extend Animal
- Multiple inheritance of type — only via interfaces (a class can implement several interfaces)

6. Polymorphism

Polymorphism (Greek for “many forms”) means the same method name or the same reference type can behave differently depending on the actual object or the arguments used. Java has two kinds of polymorphism.

6.1 Compile-Time Polymorphism (Method Overloading)

Multiple methods in the same class share a name but differ in the number or type of parameters. The compiler decides which one to call based on the arguments.

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
```

```
    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

6.2 Runtime Polymorphism (Method Overriding + Dynamic Binding)

When a parent class reference points to a child object, Java decides at runtime which overridden method to actually execute, based on the real object type, not the reference type.

```
public class Animal {
    public void makeSound() {
        System.out.println("Some generic sound");
    }
}

public class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meow");
    }
}

public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof");
    }
}

// Usage:
Animal a1 = new Cat();   // reference type Animal, real object Cat
Animal a2 = new Dog();
a1.makeSound();    // prints: Meow
a2.makeSound();    // prints: Woof
```

This is extremely powerful: you can write code such as List<Animal> animals and loop through calling makeSound() on each, without caring whether each one is a Cat, Dog, or any other Animal subtype.

7. Abstraction

Abstraction means exposing only the essential features of an object while hiding the internal implementation details. In Java, abstraction is achieved with abstract classes and interfaces.

7.1 Abstract Classes

An abstract class cannot be instantiated directly (you cannot write new Animal() if Animal is abstract). It can contain both fully implemented methods and abstract methods (methods with no body, that subclasses MUST implement).

```
public abstract class Shape {
    String color;

    // regular (concrete) method
    public void printColor() {
        System.out.println("Color: " + color);
    }

    // abstract method - no body, must be implemented by subclasses
    public abstract double calculateArea();
}

public class Circle extends Shape {
    double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

7.2 Interfaces

An interface is a fully abstract contract: a list of method signatures with no implementation (traditionally). A class uses implements to promise it will provide real code for every method in the interface. A class can implement multiple interfaces, which is how Java achieves multiple inheritance of behavior.

```
public interface Flyable {
    void fly();     // no body - implicitly public and abstract
}

public interface Swimmable {
    void swim();
}

public class Duck implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("Duck is flying");
    }
```

```
    @Override
    public void swim() {
        System.out.println("Duck is swimming");
    }
}
```

Since Java 8, interfaces can also have default methods (with a body) and static methods, giving more flexibility while keeping the core idea of a contract.

7.3 Abstract Class vs. Interface

Aspect	Abstract Class	Interface
Keyword	abstract class	interface
Instantiable?	No	No
Method bodies	Can mix abstract & concrete methods	Mostly abstract; default/static methods allowed since Java 8
Fields	Any type (instance variables allowed)	Implicitly public static final (constants only)
Inheritance	A class can extend only ONE abstract class	A class can implement MANY interfaces
Use case	Share common code among closely related classes	Define a capability/contract across unrelated classes

8. Static vs. Instance Members

By default, fields and methods belong to each object (instance) separately. Marking a member static makes it belong to the class itself, shared by all objects.

```
public class Counter {
    static int totalObjects = 0;   // shared by ALL objects
    int id;                        // separate for EACH object

    public Counter() {
        totalObjects++;
        id = totalObjects;
    }
}

// Usage:
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.totalObjects);  // 2 (shared)
System.out.println(c1.id);                 // 1
System.out.println(c2.id);                 // 2
```

Note: Access a static member using the class name (Counter.totalObjects), not an object reference, even though Java also allows the object form for readability reasons only.

9. The final Keyword

Used on...	Meaning
final variable	Value cannot be changed after initialization (a constant)
final method	Cannot be overridden by a subclass
final class	Cannot be extended / subclassed at all (e.g., String is final)

```
public final class MathConstants {   // cannot be extended
    public static final double PI = 3.14159;   // cannot be changed
}
```

10. Every Class Inherits from Object

In Java, every class you write automatically extends java.lang.Object, even if you don't write extends Object yourself. This gives every object a few built-in methods, the most important being:

- toString() — returns a text representation of the object
- equals(Object o) — defines what “equal” means for two objects
- hashCode() — returns an integer used by hash-based collections (HashMap, HashSet)

```
public class Point {
    int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Point)) return false;
        Point p = (Point) obj;
        return this.x == p.x && this.y == p.y;
    }
}
```

11. Packages

A package is a namespace/folder used to group related classes together and avoid naming conflicts. It is declared at the very top of a Java file.

```
package com.company.model;

import java.util.List;   // importing a class from another package

public class Employee {
    // ...
}
```

12. Composition vs. Inheritance (“has-a” vs “is-a”)

Inheritance models “is-a” relationships (a Dog is an Animal). Composition models “has-a” relationships, where one class contains a reference to another as a field. Composition is often preferred because it is more flexible and creates looser coupling between classes.

```
public class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

public class Car {
    private Engine engine = new Engine();   // Car HAS-A Engine

    public void start() {
        engine.start();
        System.out.println("Car is ready to drive");
    }
}
```

Note: A common design principle: “Favor composition over inheritance” — use inheritance only for genuine is-a relationships.

13. Quick Revision Cheat Sheet

Concept	One-line Definition	Java Keyword(s)
Class	Blueprint describing fields & methods	class
Object	A real instance created from a class	new
Encapsulation	Hide data, expose controlled access	private, public getters/setters

Concept	One-line Definition	Java Keyword(s)
Inheritance	Reuse code from a parent class	extends, super
Polymorphism	One name/action, many forms	overloading, @Override
Abstraction	Hide implementation, show contract	abstract, interface
Static	Belongs to the class, shared by all objects	static
Final	Cannot be changed / overridden / extended	final
Constructor	Initializes a new object	same name as class
Interface	Pure contract, multiple implementable	interface, implements

14. Practice Exercises for Students

Work through these in order. Each one builds on the previous section's concepts.

1. Create a class Student with fields name, id, and grade. Add a constructor and a method printInfo().
2. Make the fields in Student private and add public getters and setters (Encapsulation).
3. Create a class GraduateStudent that extends Student and adds a field thesisTitle (Inheritance).
4. Override printInfo() in GraduateStudent to also print the thesis title, calling super.printInfo() first (Overriding).
5. Create an interface Payable with a method calculateSalary(). Implement it in two different classes, e.g., FullTimeEmployee and PartTimeEmployee, each calculating salary differently (Abstraction + Polymorphism).
6. Create a List<Payable> containing both employee types and loop through calling calculateSalary() on each, without any if/else checking the type (Runtime Polymorphism).
7. Add a static field totalStudents to the Student class that increases every time a new student object is created.

15. Summary

OOP in Java is about modeling real-world entities as objects that combine data and behavior. Once you are comfortable creating classes and objects, focus deeply on the four pillars — Encapsulation, Inheritance, Polymorphism, and Abstraction — since virtually every Java framework (Spring, Hibernate, Android SDK) and every design pattern is built directly on top of these ideas.

The best way to truly learn OOP is to keep writing small classes yourself: model a Library, a Zoo, an OnlineStore, or a School system, and deliberately apply each of the four pillars in your design.